**ALLIANCE UNIVERSITY**
Alliance School of Advanced Computing
Computer Science and Engineering · Kalvium

MINI PROJECT REPORT

# FIVE-STAGE RISC PIPELINE SIMULATOR WITH HAZARD DETECTION

Computer Organization and Architecture
Concept: Instruction pipelining, data hazards, control hazards, and forwarding (Module 3)

*Integrated project with FLAT and DAA — one common application, three separate reports*

Submitted by
**Dasari Saideep Srinivas**
University Enrolment No. 2411021060596
Squad 70

In partial fulfilment for the award of the degree of
**BACHELOR OF TECHNOLOGY**
in
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
**Vaibhav Giri**
Technical Mentor, Kalvium × Alliance

October 2026

**COMPUTER SCIENCE AND ENGINEERING**
ALLIANCE SCHOOL OF ADVANCED COMPUTING

---

# CERTIFICATE

This is to certify that the report on Computer Organization and Architecture is submitted by Dasari Saideep Srinivas [Uni. Enrolment No. 2411021060596] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

Vaibhav Giri
Technical Mentor
Kalvium × Alliance University

Date: _____ Signature: _____

**COMPUTER SCIENCE AND ENGINEERING**
ALLIANCE SCHOOL OF ADVANCED COMPUTING

# DECLARATION

I hereby declare that the report titled FIVE-STAGE RISC PIPELINE SIMULATOR WITH HAZARD DETECTION, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Vaibhav Giri, Technical Mentor.

I further confirm that the views, findings and conclusions expressed here come from my own effort and from building and testing the simulator. This project is an integrated work shared with Formal Language and Automata Theory and Design and Analysis of Algorithms. Each subject has its own report. The COA report covers the pipeline, hazard detection, and performance metrics.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Dasari Saideep Srinivas
2411021060596

Name of the Student  University Registration Number  Signature

_____  _____  _____

## Acknowledgement

I would like to thank everyone who helped me complete this integrated microproject. This report covers the Computer Organization and Architecture component: the five-stage pipeline simulator with hazard detection.

I thank Vaibhav Giri for the project structure and for pushing us to pick concepts from Modules 1–3 that actually run in code. The COA lectures on pipelining, data hazards, control hazards, and forwarding gave me the framework. Before this project, I could draw a pipeline diagram on paper and mark where stalls go. Now I had to make the machine stall on its own, and that forced me to be precise about which stage produces a value and which stage needs it.

I also thank the faculty who taught COA Module 3. The CPI equation, ideal cycles, and Amdahl's law calculations in the statistics panel all come from those sessions.

Thank you.

Dasari Saideep Srinivas

## TABLE OF CONTENTS


Acknowledgement ..... 4
Abstract ..... 6
Chapter I – Introduction ..... 7
  Problem Statement ..... 7
  Theory ..... 8
Chapter II – System Configuration ..... 9
  Project Links ..... 9
  Pipeline Architecture ..... 9
  Source Code ..... 10
Chapter III – Results ..... 11
Chapter IV – Conclusion ..... 14
References ..... 14

## Abstract

This report covers the Computer Organization and Architecture component of an integrated microproject. The project is a RISC pipeline simulator with three subject-specific pieces. This report deals with the pipeline engine: the five stages IF, ID, EX, MEM, WB, and the hazard detection logic that decides when to stall, when to forward, and when to flush.

The simulator takes a parsed instruction stream (produced by the FLAT lexer) and executes it cycle by cycle through a classic five-stage in-order pipeline. At each cycle it checks for three kinds of hazards. Data hazards (RAW) are caught by comparing the source registers of the instruction in ID with the destination registers of instructions in EX and MEM. Load-use hazards cause a one-cycle stall because the load value is not ready until the end of MEM. Control hazards from taken branches flush the IF and ID slots. Forwarding, when enabled, lets the pipeline avoid most stalls by routing the ALU result from EX or the memory data from MEM back to the ID stage.

The output is a cycle-by-cycle Gantt chart, a set of performance metrics (CPI, pipeline speedup over sequential execution, stall count, forward count), and the final register file. The user can toggle forwarding on and off to see how many extra stalls appear without it. That toggle is the core demonstration: it shows exactly why forwarding matters, not as a textbook claim but as a number the user can compare.

## Chapter I – Introduction

Pipelining is the central idea of COA Module 3. Instead of finishing one instruction before starting the next, the processor overlaps them. While instruction I1 is in the Execute stage, I2 is being decoded and I3 is being fetched. In the ideal case, one instruction completes every cycle after the pipe fills, giving a CPI close to 1 and a throughput close to the clock rate.

The catch is hazards. A data hazard happens when an instruction needs a value that a previous instruction has not yet written. A control hazard happens when a branch changes the program counter and the instructions already fetched after the branch are wrong. A structural hazard happens when two stages need the same resource. These hazards break the ideal CPI. The COA response is forwarding for data hazards and branch prediction or flushing for control hazards.

I chose this topic because it is the heart of Module 3 and because it connects naturally to the other two subjects. The pipeline needs a token stream (FLAT provides the lexer), and the forwarding decision is a greedy scheduling choice (DAA provides the analysis). Together they form one application where each subject has a visible, testable role.

### Problem Statement

Given a sequence of RISC instructions, simulate their execution through a five-stage pipeline (IF → ID → EX → MEM → WB). Detect data hazards (RAW), load-use hazards, and control hazards. When forwarding is enabled, route values from EX or MEM back to ID to avoid stalls where possible. Produce a cycle-accurate Gantt chart showing which instruction is in which stage at each cycle, and compute CPI, total stalls, and pipeline speedup.

### Theory

The five stages are Instruction Fetch (IF), Instruction Decode / register read (ID), Execute (EX), Memory access (MEM), and Write Back (WB). In the ideal case with n instructions, the pipeline completes in n + 4 cycles (four being the number of extra stages needed to fill the pipe).

A Read After Write (RAW) hazard occurs when instruction Ij reads register Rx and an earlier instruction Ii writes Rx, and Ii has not yet reached WB. Without forwarding, the pipeline must stall Ij until Ii finishes WB. With forwarding (also called bypassing), the value can be sent from the EX or MEM stage of Ii to the ID stage of Ij, avoiding the stall. The one exception is the load-use case: if Ii is a load (LW), the data is not available until the end of MEM, so a one-cycle stall is unavoidable even with forwarding.

A control hazard occurs when a branch is taken. In a static predict-not-taken scheme, the instructions fetched after the branch are wrong and must be flushed. The branch is resolved in EX, so the IF and ID slots are flushed when EX detects a taken branch.

CPI is actual cycles divided by instruction count. Pipeline speedup over sequential execution is (n × 5) / actual cycles, where 5 is the number of stages each instruction would take without pipelining. These formulas come directly from COA Module 3, LU 3.14.

## Chapter II – System Configuration

Same platform as the FLAT report: a static web application in JavaScript, running in Chrome on Windows. No build tools. The pipeline engine is in js/pipeline.js. It reads the instruction array produced by the lexer and parser in js/lexer.js.

### Project Links

GitHub repository: open to view

Live deployment: open to view

### Pipeline Architecture

The pipeline is modelled as an array of five slots, one per stage. Each slot holds a reference to the instruction, the current stage label, ALU result, memory data, forwarding annotations, and a flushed flag.

At each cycle, the simulator processes stages in reverse order (WB first, then MEM, EX, ID, IF) to avoid write-before-read conflicts. This is the standard simulation trick: by processing WB first, the register write happens before the next IF reads the register file.

**Hazard detection (ID stage):**

1. Get the source registers of the instruction in ID.
2. Compare each source with the destination register of the instruction in EX. If they match:
    - If the EX instruction is a LOAD → stall (load-use hazard).
    - Otherwise → forward from EX (EX→EX forwarding path).
3. Compare each source with the destination register of the instruction in MEM. If they match → forward from MEM (MEM→EX forwarding path).
4. If a stall is needed, insert a bubble into EX and freeze IF and ID for one cycle.

### Source Code

The core hazard check function from *pipeline-sim/js/pipeline.js*:

```
function checkHazards(idSlot, exSlot, memSlot) {
  const result = { stall: false, forwards: [] };
  const srcs = srcRegs(idSlot.insn)
               .filter(r => r !== undefined && r !== 0);
  for (const src of srcs) {
    // EX stage check
    if (exSlot && dstReg(exSlot.insn) === src) {
      if (exSlot.insn.format === 'LOAD') {
        result.stall = true;         // load-use
        return result;
      }
      result.forwards.push(
        { from:'EX', to:'ID', reg:src }  // EX→EX fwd
      );
      continue;
    }
    // MEM stage check
```

```
    if (memSlot && dstReg(memSlot.insn) === src) {
      result.forwards.push(
        { from:'MEM', to:'ID', reg:src } // MEM→EX fwd
      );
    }
  }
  return result;
}
```

## Chapter III – Results

**Test 1: Data hazard with forwarding**

Program: ADD R1, R0, R2 / ADD R3, R1, R4 / SUB R5, R3, R1 / HLT

The second instruction reads R1, which the first writes. With forwarding enabled, the value is forwarded EX→EX and no stall is inserted. Without forwarding, a 2-cycle stall appears.

| | C1 | C2 | C3 | C4 | C5 | C6 | C7 |
|---|---|---|---|---|---|---|---|
| ADD R1,R0,R2 | IF | ID | EX | MEM | WB | | |
| ADD R3,R1,R4 | | IF | ID | EX | MEM | WB | |
| SUB R5,R3,R1 | | | IF | ID | EX | MEM | WB |

With forwarding: 7 cycles, CPI = 7/3 ≈ 2.33 (including pipeline drain). The forwarding log shows EX→EX fwd R1 at cycle 3 and EX→EX fwd R3 at cycle 4.

**Test 2: Load-use stall**

Program: LW R1, 0(R2) / ADD R3, R1, R4 / HLT

ADD needs R1 from the load, but the data is not ready until MEM completes. Even with forwarding, a 1-cycle stall is inserted.

| | C1 | C2 | C3 | C4 | C5 | C6 | C7 |
|---|---|---|---|---|---|---|---|
| LW R1,0(R2) | IF | ID | EX | MEM | WB | | |
| BUBBLE | | | | — | | | |
| ADD R3,R1,R4 | | IF | ID | ID | EX | MEM | WB |

Stall count: 1. This matches the textbook prediction for a load-use hazard with forwarding.

**Test 3: No hazards**

Program: ADD R1, R2, R3 / SUB R5, R6, R7 / AND R8, R9, R10 / OR R11, R12, R13 / HLT

No register overlaps. 0 stalls, 0 forwards. CPI = 9/5 = 1.80. Pipeline speedup vs sequential = (5×5)/9 = 2.78×. That approaches the ideal of 5× as the instruction count grows.

**Test 4: Branch flush**

A BEQ that is taken causes the IF and ID slots to be flushed. Control hazard count increments by 1. The Gantt chart shows FLUSH labels on the two discarded instructions.

**Summary of metrics across tests**

| Test | Instructions | Cycles (fwd on) | Stalls | Forwards | CPI | Speedup |
|---|---|---|---|---|---|---|
| Data hazard | 4 | 8 | 0 | 2 | 2.00 | 2.50× |
| Load-use | 3 | 8 | 1 | 0 | 2.67 | 1.88× |
| No hazards | 5 | 9 | 0 | 0 | 1.80 | 2.78× |
| Branch | 5 | 10 | 0 | 1 | 2.00 | 2.50× |
| Complex mix | 9 | 14 | 1 | 4 | 1.56 | 3.21× |

The "Complex mix" test is the most realistic. It has a load, a store, ALU chains, and immediates. CPI of 1.56 is close to the ideal 1.0 for a long program, and the speedup of 3.21× over sequential shows that pipelining with forwarding is doing its job.

## Chapter IV – Conclusion

The pipeline simulator works as described. It takes a stream of parsed instructions, runs them through five stages, detects data hazards and control hazards, inserts stalls or forwards values as needed, and produces a Gantt chart with accurate cycle counts. Toggling forwarding off increases stalls, which is exactly what the theory predicts.

The COA concept — instruction pipelining with hazard detection from Module 3 — is the largest piece of this integrated project. Every other component (the FLAT lexer, the DAA scheduling logic) feeds into or builds on the pipeline. Without the pipeline, there is nothing to forward and nothing to stall.

What I understood better after building this is the timing. On paper, I knew that a load-use hazard causes one stall. In the simulator, I had to figure out that the stall means the EX stage gets a bubble, the ID and IF slots freeze, and the MEM and WB stages keep draining. Getting that wrong by one cycle breaks the Gantt chart. That precision is what separates knowing the concept from implementing it.

A limitation is that the simulator is in-order and single-issue. Superscalar or out-of-order execution (COA Module 5) would need a different engine. But for the scope of Module 3, the five-stage in-order model is complete.

## References


1. Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design: The hardware/software interface* (RISC-V ed.). Morgan Kaufmann. (Chapters 4.5–4.8: Pipelining, hazards, forwarding.)
2. Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann. (Appendix C: Pipelining.)
3. Stallings, W. (2022). *Computer organization and architecture: Designing for performance* (11th ed.). Pearson. (Chapter 14: Instruction-level parallelism.)